

SELF-SERVICE REPORTING

Report Builder

Proof of Concept - Technical Walkthrough

~6.8 developer days (6.8 required + 0 optional polish)

What this is

This is a proof of concept (POC) of the Report Builder: a stripped-down, demo-able clone of the full feature built to put the core experience in front of the client quickly. The POC proves the experience - drag tables onto a flow canvas, pick columns, connect joins, then refine the report in plain language with an AWS Bedrock assistant, preview it, run it, and download a CSV.

The one-line pitch: "Build a report by dragging tables and picking columns - then just tell the assistant what you want, and watch it change." The demo is about the experience, not the plumbing.

It runs as a single demo user against a single configured demo scope (one demo bryt number) on a demo dataset. It deliberately carries no governance/security spine and no production infrastructure - those are the full spec's job on green-light.

The POC line, in one sentence

The POC builds the parts a client can see and feel - the canvas, the column picker, and above all the assistant - on the simplest plumbing that demos reliably; everything that makes the feature safe and production-grade is deferred to the full build on green-light.

What the POC keeps vs defers

A clear line runs through the whole POC: keep what proves the experience, defer what makes it production-safe.

KEPT FOR THE DEMO	DEFERRED TO THE FULL BUILD
Builder canvas - drag tables, connect joins visually	The entire governance & security spine (per-customer isolation, independent Query_Verifier, output verification)
Column picker - choose exactly what each table contributes	Prompt-injection defence, configurable query bounds, audit logging
Assistant drawer (Bedrock Converse) - the star of the demo	Bryt-number resolution + admin override; JWT auth; multi-tenant identity

KEPT FOR THE DEMO	DEFERRED TO THE FULL BUILD
Preview, Run → download CSV, light My Reports / Save	Production infra: Step Functions pipeline, DynamoDB single-table, versioned/encrypted S3, per-env IAM + Lake Formation role
Shared Report_Design model + Catalog / Join_Manifest	Run cancellation, deep history, presigned owner-scoped downloads, conversation persistence

How the POC works

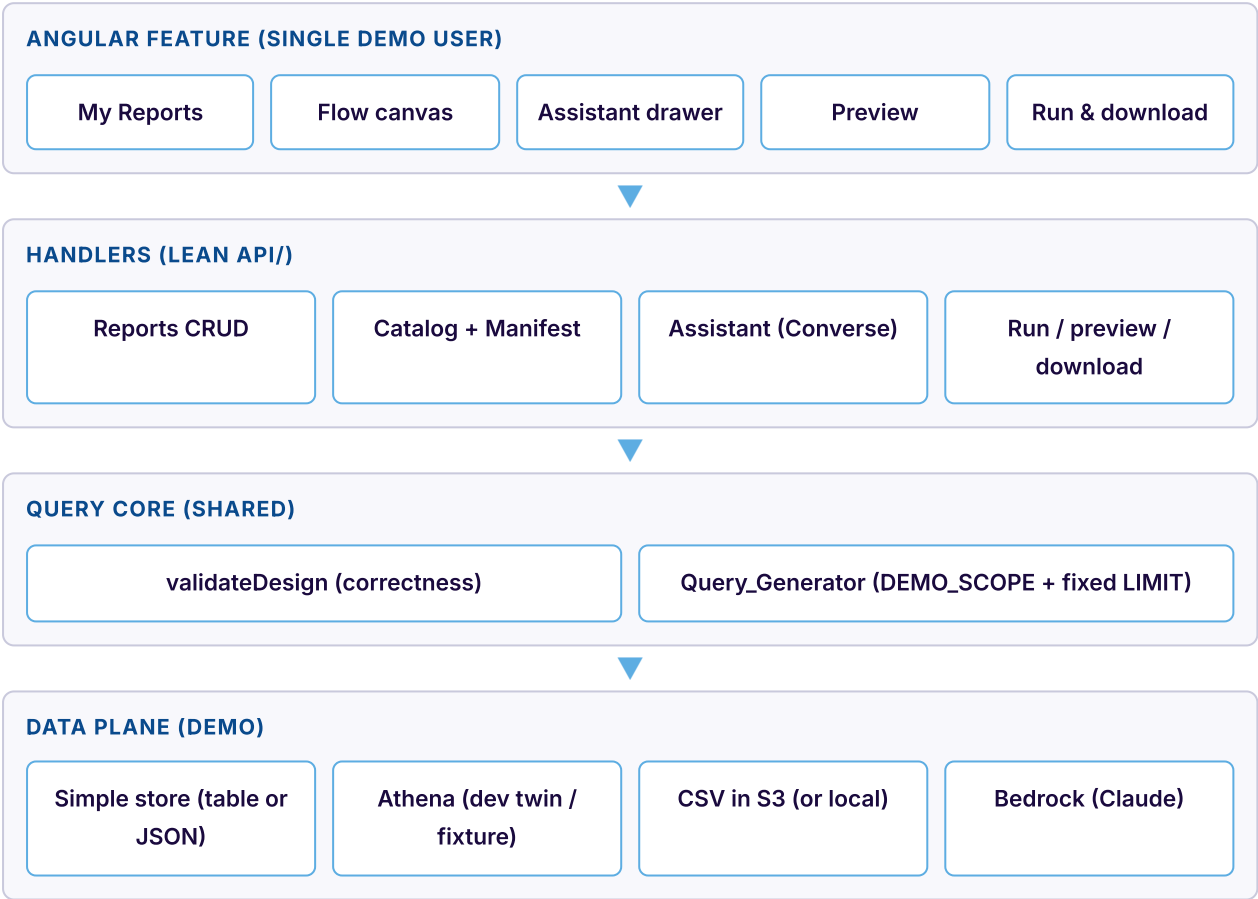
A report is captured as a Report_Design - a logical description of intent (selected tables, columns, joins, filters, sort), never SQL. The same design instance is edited by both the flow canvas and the assistant, so visual and conversational edits stay in sync. This is the same shape as the full spec, so the POC's core pieces carry forward into the production build rather than being thrown away.

SQL is derived from the design only at run or preview time, by the Query_Generator, which scopes every query to a single DEMO_SCOPE constant and a fixed LIMIT. In the POC the demo scope is a demo convenience, not a security boundary - there is no independent verifier and no per-request identity resolution. Those are exactly the pieces the full build layers on top.

The backend is a lean api/ + web/ project rather than the strict production repo layout. Reports live in the simplest store that works (one table or local JSON); runs go straight to Athena and land as CSV in S3 (or locally for a local demo) - no Step Functions.

POC architecture (simplified)

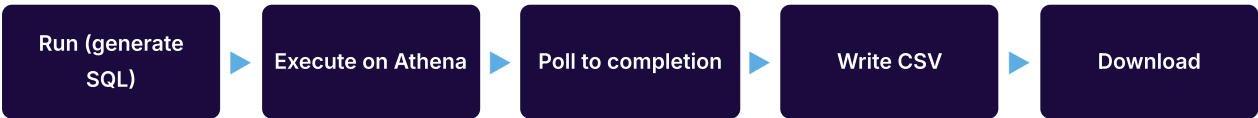
The Angular feature talks to a small set of handlers. There is no identity/authorisation lane and no independent verifier - the Query_Generator applies the single DEMO_SCOPE and a fixed LIMIT, and queries run directly against Athena.



No identity lane, no independent verifier, no Step Functions - the POC substitutes one DEMO_SCOPE constant for the whole isolation stack.

Run flow (no Step Functions)

Running a report generates the DEMO_SCOPE-scoped SQL, executes it on Athena polled to completion, writes the CSV, and offers it for download. Preview runs the same generate path with a bounded LIMIT synchronously, without queuing a run. States are Queued → Running → Complete / Failed - there is no Cancelled state in the POC.

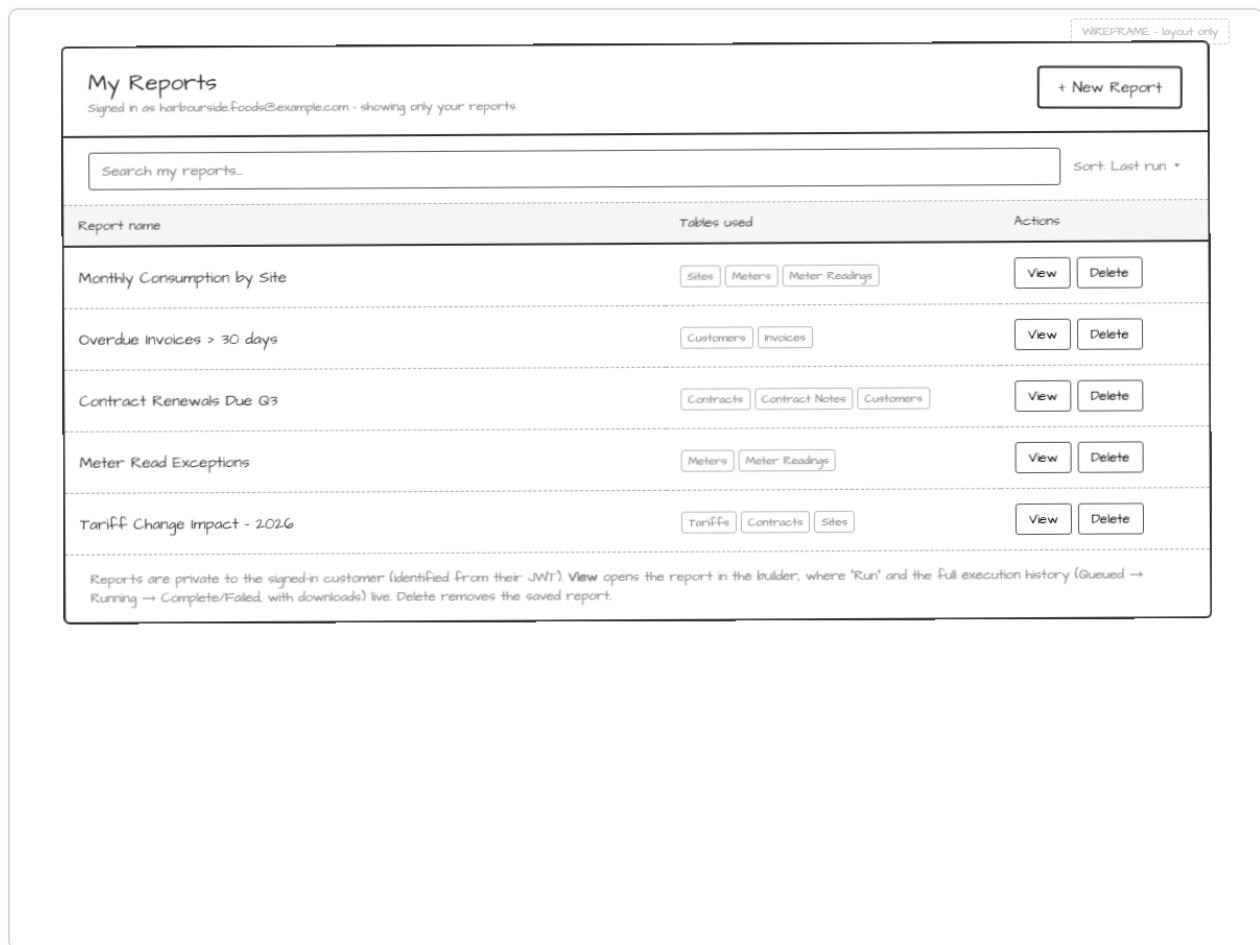


A lean, synchronous-feeling pipeline: no queue, no Step Functions, no cancellation - just enough to end the demo on a real, downloadable result.

Screen-by-screen walkthrough

The POC reuses the full feature's screen mockups. Seven screens make up the demo, from a light reports list through the visual builder, the assistant, preview, run, and save. The notes below describe each screen at POC fidelity - single demo user, one demo scope, no auth or verifier behind it.

1. My Reports



The entry point: a light list of saved demo reports, each showing the tables it uses, with search and sort. In the POC this is a single demo user's list, not a private per-customer view.

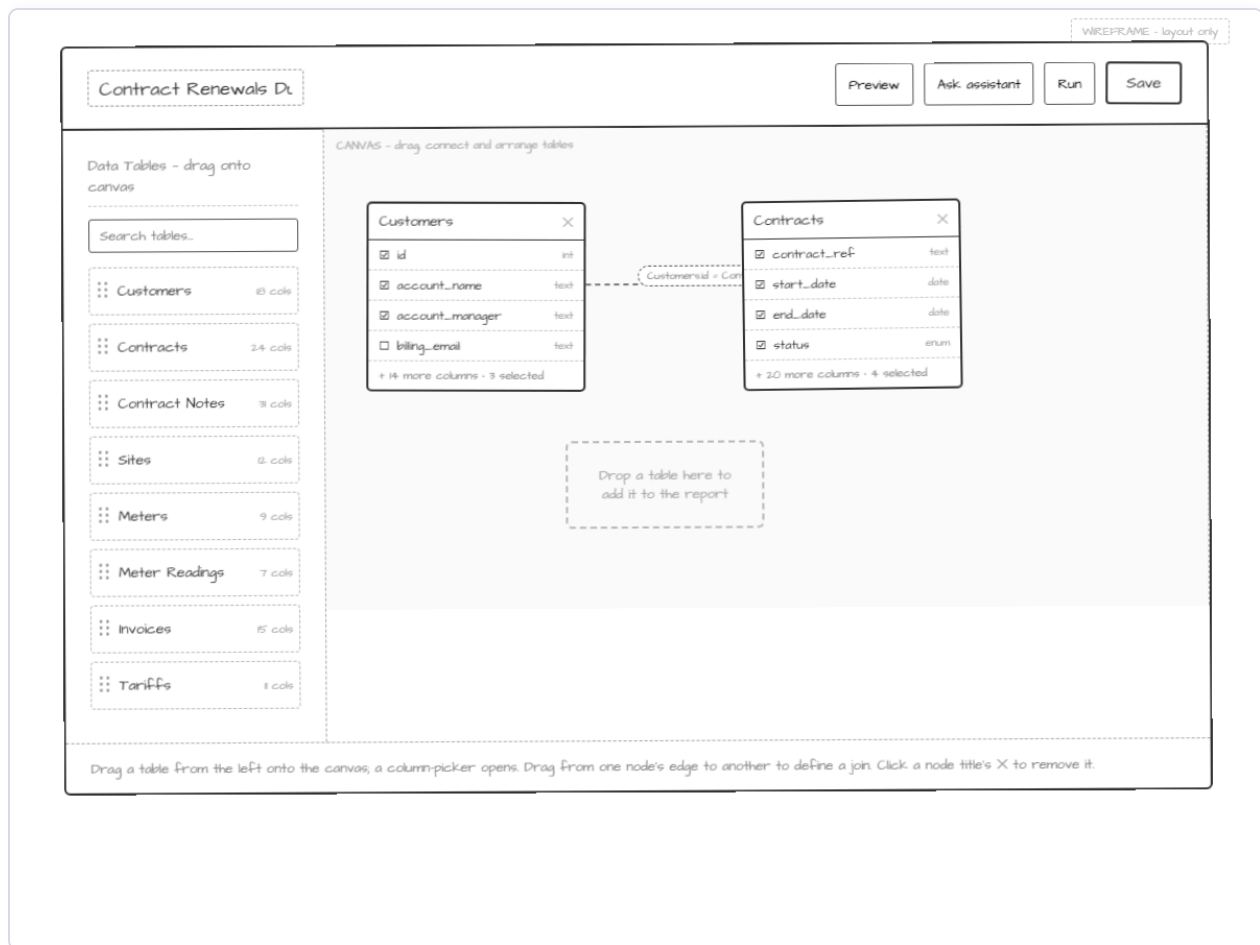
KEY INTERACTIONS

- › + New Report opens the builder with an empty design
- › View opens a saved report's design in the builder
- › Delete removes a report; search + A-Z / Z-A sort help find one
- › Empty state invites creating the first report

BEHIND THE SCREEN

- › Reports come from the simple store (one table or local JSON), not an owner-scoped DynamoDB partition
- › No JWT / identity check - the demo runs as one Demo_User
- › Search and sort are done client-side over the small demo set

2. Builder Canvas



The visual designer and the heart of the no-SQL promise. A palette of allow-listed tables sits on the left; dragging a table onto the canvas opens the column picker, and dragging between node edges creates a join from the manifest predicate.

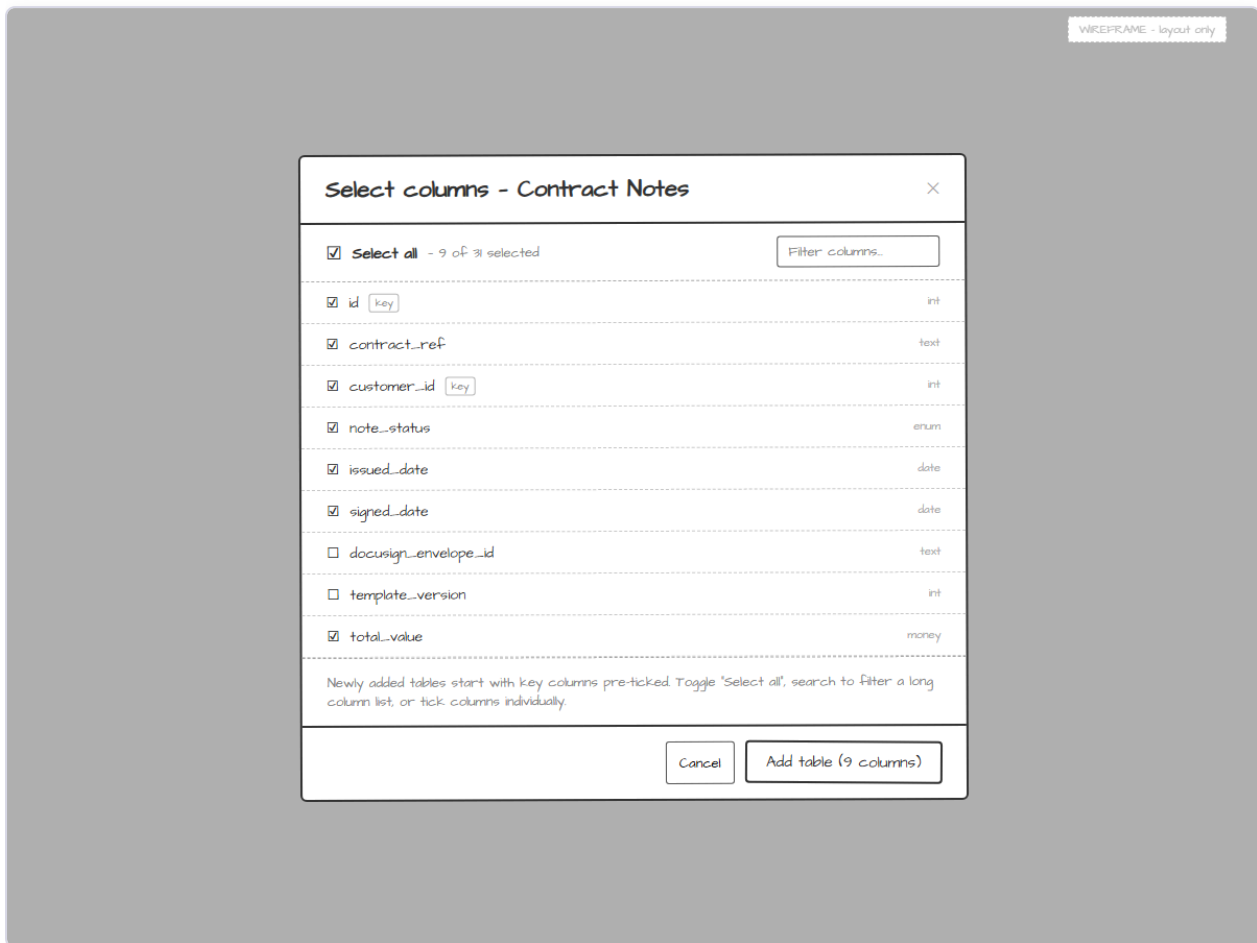
KEY INTERACTIONS

- › Drag a table from the palette → opens the column picker
- › Drag from one node's edge to another - > creates a manifest-defined join
- › Remove a node to drop the table and its joins from the design
- › Header actions: Preview, Ask assistant, Run, Save

BEHIND THE SCREEN

- › Each node is one `SelectedTable`; each edge is one `DesignJoin` (1:1 graph mapping)
- › Every edit runs `validateDesign` against the catalog + manifest (a correctness check, not a security gate)
- › The minimal demo table set keeps the query surface small and the demo reliable

3. Column Picker



Opened when a table is added. Lists the allow-listed columns with type and a key tag; key columns are pre-selected, everything else starts unselected.

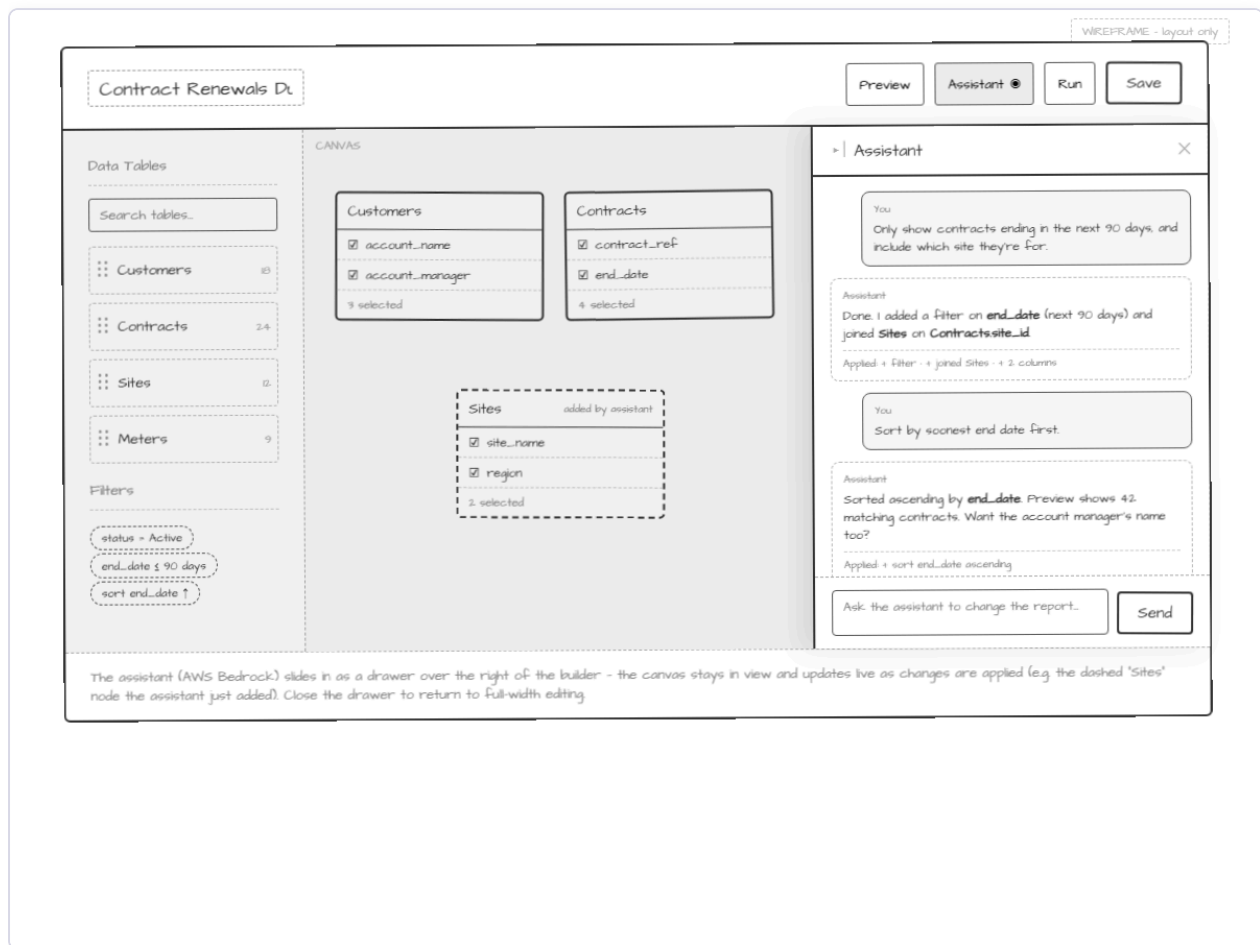
KEY INTERACTIONS

- › Toggle individual columns; the 'X of N selected' count updates live
- › Select all (respects an active filter), and filter by column name
- › Add table confirms with at least one column selected

BEHIND THE SCREEN

- › Columns come from the static/cached catalog allow-list, with type and isKey flags
- › Key columns are pre-selected on first open
- › Confirming with zero columns is blocked with a message

4. Assistant Drawer - the star



A Bedrock-backed helper that reads and edits the same Report_Design in response to plain-language requests, describing each change it applies. This is the centrepiece of the demo, prioritised over everything else.

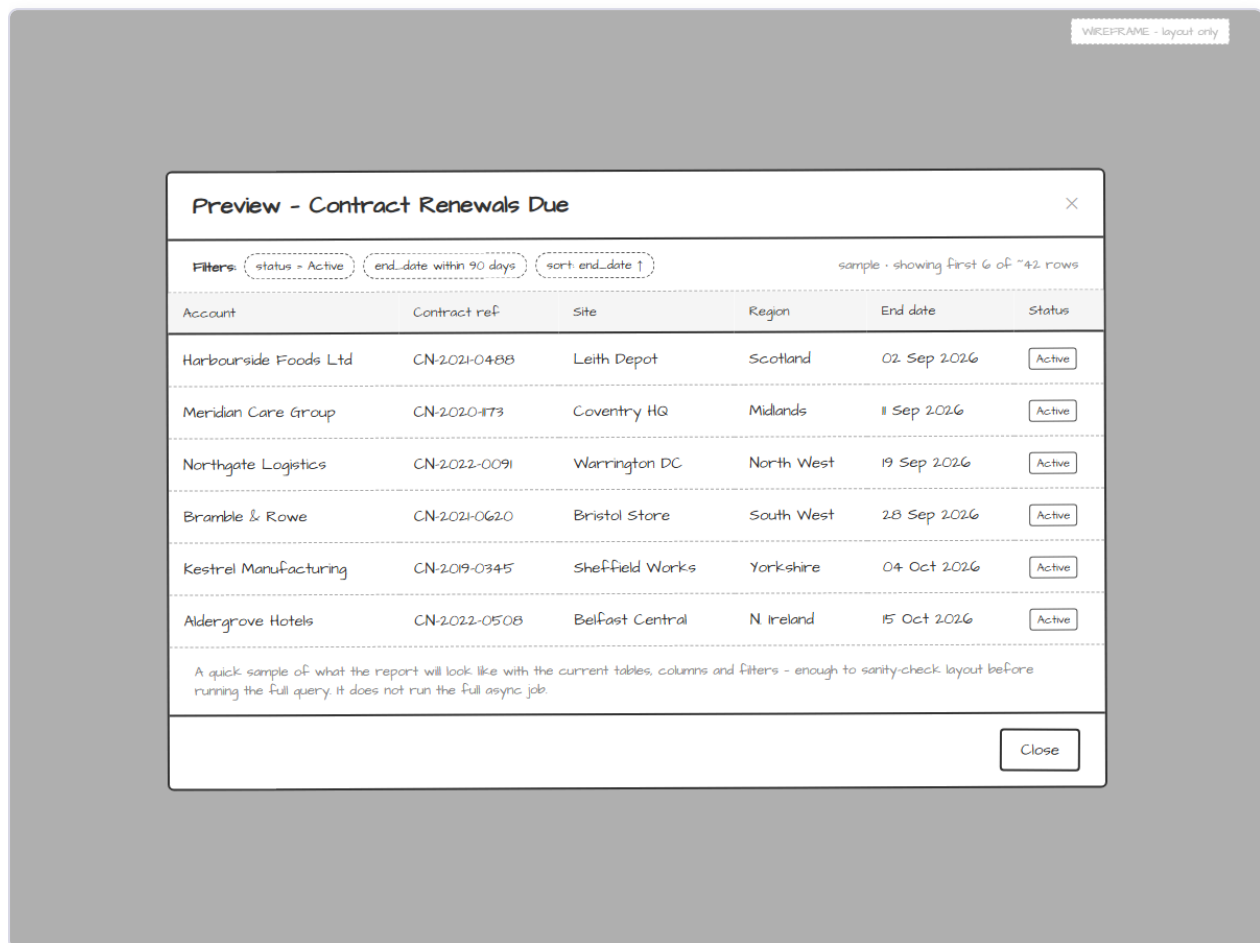
KEY INTERACTIONS

- › Submit a plain-language message (e.g. 'only contracts ending in the next 90 days, and add the site')
- › Applied changes update the shared design and reflect on the canvas live
- › The assistant returns a per-change description and an applied-change summary
- › Closing the drawer returns to full-width editing with the design retained

BEHIND THE SCREEN

- › A Bedrock Converse tool-use loop (Claude) with Report_Design mutation tools (add_table, add_join, set_filter, ...)
- › Mutations run through the same validateDesign the canvas uses
- › POC-only: no prompt-injection defence, no forced validate_query, no audit - all deferred to the full build
- › Conversation history is not persisted in the POC

5. Preview



A lightweight sample of the output to sanity-check layout before running the full job. Preview does not queue a run.

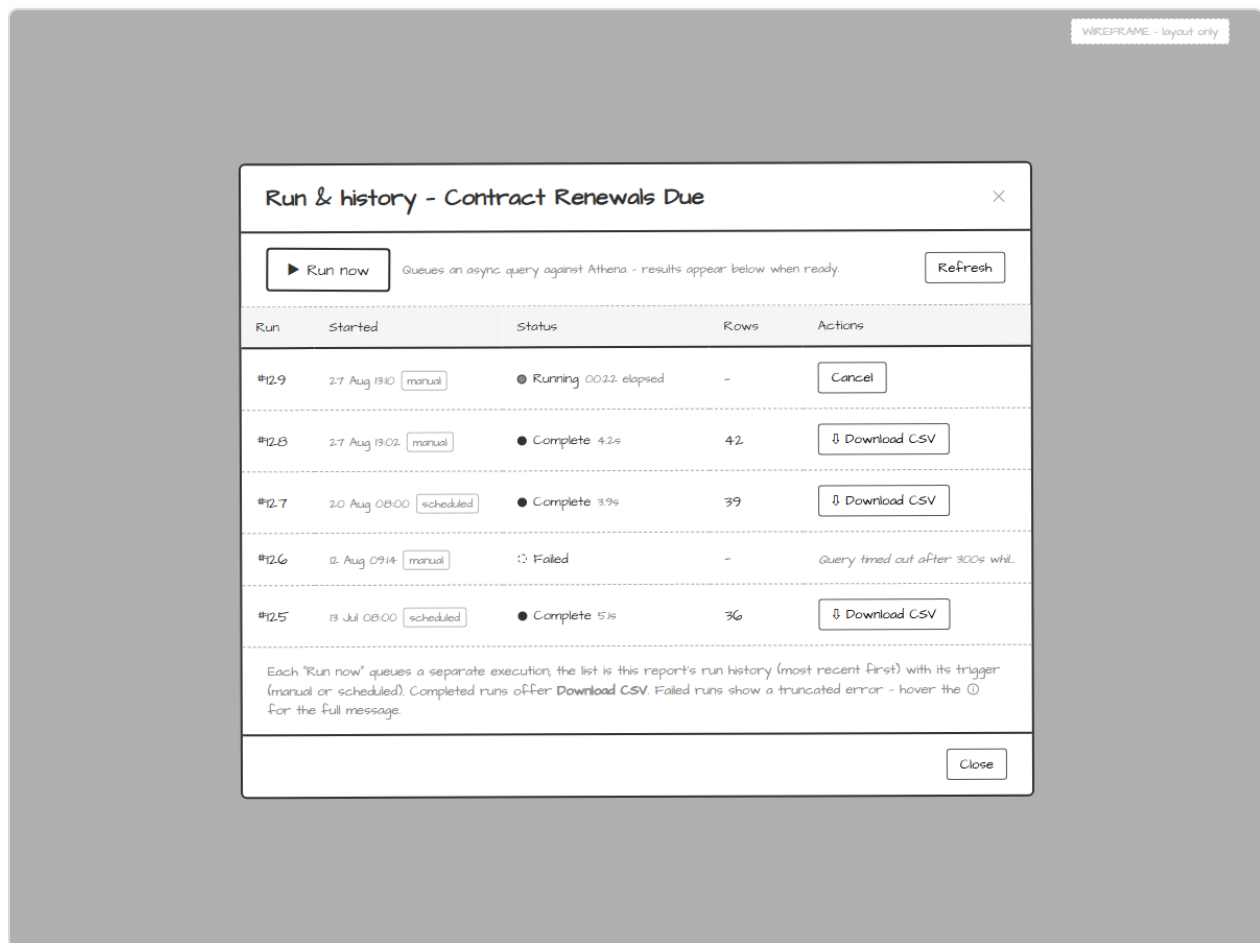
KEY INTERACTIONS

- › Preview shows a bounded sample, only the selected columns, in design order
- › A filter/sort summary strip lists the active rules
- › A progress indicator shows while generating; Close returns to the builder

BEHIND THE SCREEN

- › Runs the same generate path with a bounded LIMIT, scoped to DEMO_SCOPE, synchronously
- › No asynchronous run is queued or started
- › Optional EXPLAIN dry-run can fail fast (polish, not a gate)

6. Run & Download



Runs the report against Athena and lets the user download the CSV result - the tangible finish to the demo.

KEY INTERACTIONS

- › Run generates the SQL, executes on Athena, and shows progress to completion
- › A completed run offers Download CSV
- › States shown: Queued → Running → Complete / Failed

BEHIND THE SCREEN

- › No Step Functions - the handler generates, runs Athena polled to completion, and writes the CSV
- › Download is a straight fetch of the CSV (no presigned owner-scoped URL in the POC)
- › No Cancelled state and no deep run history

7. Save Report

WIREFRAME - layout only

Save report

Report name

Contract Renewals Due

Description (optional)

Active contracts ending within 90 days, with site and region. Used for the quarterly renewals review.

Tables used

Customers Contracts Sites

Saved reports appear in 'My Reports' and can be run or re-opened in the builder at any time.

Cancel Save report

Persists the design under a name so a report can be saved and reopened live during the demo, with the tables used shown as badges.

KEY INTERACTIONS

- › Confirm save with a valid name persists the report
- › An empty/whitespace name is blocked with a message
- › A saved report appears back in My Reports; saving an existing report updates it in place

BEHIND THE SCREEN

- › The serialised Report_Design is stored in the simple store
- › No versioned S3 snapshot / history in the POC

Demo setup & data

The POC is a single-tenant demo by construction. These are the working choices that keep it simple and reliable in front of the client.

AREA	POC CHOICE
Identity	One Demo_User; no JWT, no per-request identity resolution, no admin override
Scoping	A single DEMO_SCOPE constant (one demo bryt number) applied by the Query_Generator, plus a fixed LIMIT
Data source	Dev twin dev_esg_ci_data_eng_master_record_db (eu-west-2) or a small fixture - whichever demos most reliably
Table surface	The minimal demo table set, preferring direct-pinned tables to keep the query surface small
Assistant	Bedrock Converse tool-use (Claude) per the shared bedrock-approach - minus injection defence / audit / forced-validate
Persistence	The simplest store that works: one table or local JSON
Run execution	Query_Generator → Athena directly; CSV in S3 (or local) - no Step Functions

Delivery breakdown

The POC build is grouped into six phases. Figures are indicative, derived from the 18-task POC plan, and are pulled live from the estimate spreadsheet - the assistant (Phase 3) is prioritised as the star of the demo.

PHASE	SCOPE	DAYS
Foundations	Lightweight api/ + web/ scaffold, shared Report_Design domain types (same shapes as the full spec), Join_Manifest + minimal demo table set, the single DEMO_SCOPE + fixed LIMIT	0.8
Query generation + validation	validateDesign (allow-list + manifest correctness check, not a security gate), Query_Generator (design → Athena SQL scoped to DEMO_SCOPE, fixed LIMIT, bound params), Report_Design serialise round-trip	0.8

PHASE	SCOPE	DAYS
Backend services	Catalog service (static/cached allow-list + manifest), Reports CRUD against a simple store, the Bedrock Converse assistant loop + mutation tools (the star), optional EXPLAIN dry-run polish	1.2
Run + preview + download	Run handler (generate → Athena → CSV, polled, no Step Functions), CSV download, synchronous bounded preview	0
Frontend	Flow-canvas library spike, client Report_Design + graph mapping, the seven demo screens for a single demo user	3.5
Demo readiness	Seed demo reports + data, script the assistant moment, smooth repeatable end-to-end run-through	0.5
Total	18 tasks across 6 phases	6.8

After the demo - the green-light path

On client approval we do not harden this POC. We execute the full report-builder spec, which already carries the security spine and production infrastructure. The POC's kept pieces - the Report_Design model, Query_Generator, Catalog/Join_Manifest, and the Converse assistant - are a genuine seed for that build, so the work here is not thrown away. The full feature, with its own requirements, design, task plan, and estimate, lives alongside this POC in the report-builder spec.